



Tecnológico Nacional de México

Instituto Tecnológico de Tepic

Ingeniería en Sistemas Computacionales

Laboratorio de Servicios Web

Unidad 2: Infraestructura Web de Lenguaje de Definición de Esquemas - GraphQL API (SDL)

T2-Actividad 2. Integrar en la aplicación de Express el desarrollo de las APIs RESTful del proyecto final para los CRUDs de las tablas (Supabase) y colecciones (MongoDB) necesarias

Equipo 4 · Integrantes:

Ocegueda Ruelas Antony Daniel · 22400630

Pérez Solís Kevin Alberto · 20400798

García Vázquez Edgar Daniel · 22400591

Garnica Talavera Gustavo Adolfo · 20400743

Cárdenas Gámez Samir Adrián · 20400712

Camarena Martínez José Alfredo · 21400657

Docente: Dr. Francisco Ibarra Carlos

Grupo: 5A

T2-Actividad 2. Integrar en la aplicación de Express el desarrollo de las APIs RESTful del proyecto final para los CRUDs de las tablas (Supabase) y colecciones (MongoDB) necesarias.

A) Realizar un listado de todas las APIs a desarrollar para cada uno de los submódulos del proyecto integrador. Explique para qué servirá cada una de ellas, cuál es la ruta con la que se mandará llamar, qué información se enviará en el request (parámetros que recibe tanto en header como en body) y qué información regresará en el response.

Nota 1: sobre todo tomar en cuenta las APIs que insertarán, modificarán y eliminarán un solo registro o subdocumento (objeto) o varios registros o subdocumentos (arreglo de objetos).

Nota 2: la nota compartida de Evernote contiene por cada equipo una tabla donde se describe esta información; esta tabla debe agregarse en la evidencia de la actividad.

B) Crear la estructura base del proyecto final: en la aplicación del proyecto creada en la actividad anterior verificar su configuración para integrar las APIs del submódulo que le tocó desarrollar a cada equipo.

C) Crear y configurar las "clases modelo" para todas y cada una de las APIs del submódulo asignado al equipo.

D) Crear o complementar la configuración de las conexiones a las bases de datos SQL (Supabase) y NoSQL (MongoDB) en las variables de entorno. Las APIs deberán operar con la base de datos seleccionada al iniciar sesión.

E) Crear los "Servicios" para todas las APIs, métodos GET, POST, PUT y DELETE del submódulo, no solo por la colección principal sino por cada subdocumento.

F) Crear los "Controladores" para todas las APIs, métodos GET, POST, PUT y DELETE, por la colección principal y por cada subdocumento.

G) Crear los "Ruteos" para todas las APIs, métodos GET, POST, PUT y DELETE, por la colección principal y por cada subdocumento.

H) Configurar el "Index" y "App" principal para la ejecución y despliegue de la Aplicación RESTful API.

Índice

Índice 3
1. Introducción 5
2. Objetivos 5
    2.1 Objetivo general 5
    2.2 Objetivos específicos 5
3. Listado de APIs del submódulo 6
    3.1 Convenciones comunes 6
    3.2 Listado general 6
    3.3 Request y response de cada API 7
4. Estructura base del proyecto final 9
    4.1 Qué se verificó y qué se agregó 9
    4.2 Arquitectura del submódulo 9
    4.3 Estructura de carpetas 10
5. Clases modelo 10
    5.1 Clase modelo en MongoDB 10
    5.2 Clase modelo en Supabase 11
6. Conexiones a Supabase y MongoDB con variables de entorno 13
    6.1 Variables de entorno 13
    6.2 Configuración centralizada 14
    6.3 Conexión a Supabase 14
    6.4 Selección de la base al iniciar sesión 15
7. Servicios 16
    7.1 Reglas de negocio 17
    7.2 Servicios de la colección principal 17
    7.3 Servicios del subdocumento 18
    7.4 Repositorios: donde sí se nota cada motor 19
8. Controladores 20
9. Ruteos 21
10. Configuración del Index y del App principal 23
    10.1 index.js 23
    10.2 Registro de rutas 23
    10.3 Arranque 23
11. Pruebas y evidencias 24
    11.1 Datos de prueba 24
    11.2 Prueba automatizada en ambos motores 24
    11.3 Evidencias con Supabase 26
    11.4 Evidencias con MongoDB 28
    11.5 Manejo de errores 31
    11.6 Un solo contrato para los dos motores 31

11.7 Problemas encontrados y cómo se resolvieron **31**
12. Repositorio y bitácora **32**
13. Conclusiones **33**
Referencias **33**



1. Introducción

In the first unit the team built the base structure of a RESTful application in Express, using the High School Catalog as a DEMO process. This activity takes that same application and turns it into the home of the final project: the team now integrates the real APIs of the submodule it was assigned, the Catalog of Catalogs, which stores the master data (brokers, strategies, indicators and timeframes) that the rest of the system reads.

The key requirement is that the same API must work on two very different databases. Supabase keeps the data in relational tables, while MongoDB keeps each catalog as a single document with an embedded array of values. The user chooses the database when logging in, and from that moment every request is resolved against that engine without changing a single route.

The document follows the order requested by the instructor: first the list of APIs with their routes, requests and responses; then, layer by layer, the models, the connections, the services, the controllers, the routes and the main index and app; and finally the tests that show both engines returning exactly the same contract.

2. Objetivos

2.1 Objetivo general

Integrar en la aplicación de Express del equipo (app_restful_express) las APIs RESTful del submódulo Catálogo de Catálogos del proyecto final, con operaciones CRUD sobre las tablas de Supabase y las colecciones de MongoDB, de modo que cada petición se resuelva en la base de datos elegida al iniciar sesión.

2.2 Objetivos específicos

- Documentar todas las APIs del submódulo: propósito, ruta, request (header, parámetros y body) y response.
- Verificar y extender la estructura base del proyecto para alojar el submódulo sin romper lo construido en la T1.
- Crear las clases modelo para MongoDB (colección con subdocumento) y para Supabase (tablas maestro y detalle).
- Configurar las conexiones a Supabase y MongoDB en variables de entorno y elegir la base al iniciar sesión.
- Implementar servicios, controladores y ruteos para GET, POST, PUT y DELETE, por la colección principal y por el subdocumento, para uno y para varios elementos.
- Configurar el Index y el App principal, documentar con Swagger y comprobar el funcionamiento con pruebas en ambos motores.

Descripción de la actividad

Desarrollar e integrar las APIs RESTful del proyecto final en Express, definiendo sus rutas, parámetros y respuestas, así como la estructura del proyecto, los modelos, servicios, controladores y ruteos necesarios para implementar las operaciones CRUD en Supabase y MongoDB, configurando las conexiones a las bases de datos y los archivos principales para la correcta ejecución de la aplicación.

3. Listado de APIs del submódulo

El inciso A pide listar todas las APIs a desarrollar para el submódulo del proyecto integrador. Al Equipo 4 le corresponde el **Catálogo de Catálogos**: un modelo de datos maestros que sustituye las muchas tablas pequeñas de valores (estatus, tipos, giros, niveles) por una estructura genérica. Cada catálogo tiene una llave de negocio (`key`) y una lista de valores; cada valor puede tener un padre (`parentId`), lo que permite tres o más niveles (maestro-detalle-detalle).

En Supabase esto se guarda en dos tablas, `catalogs` y `catalog_values`; en MongoDB, en una sola colección `catalogs` donde cada documento trae sus valores embebidos en el subdocumento `values[]`. Por eso las APIs se agrupan en dos familias: las de la colección o tabla principal y las del subdocumento o tabla detalle.

3.1 Convenciones comunes

Todas las APIs comparten las mismas reglas, lo que evita repetirlas en cada renglón de las tablas siguientes:

Tabla 1. Convenciones comunes a todas las APIs del submódulo.

Aspecto	Convención
URL base	`http://localhost:3050/api/v1` (puerto y prefijo vienen del .env)
Sesión	Primero `POST /auth/login` con `{ usuario, dbServer }`. La respuesta trae un token JWT que guarda la base elegida.
Header de la petición	`Authorization: Bearer <token>` (obligatorio en todas las rutas de catálogos) y `Content-Type: application/json` cuando hay body.
Header de la respuesta	`X-Db-Server: mongodb | supabase`, para saber en qué base se resolvió la petición.
Formato	JSON en camelCase en ambos motores. Supabase guarda snake_case y el repositorio traduce.
Errores	`{ statusCode, error, message }` con 400, 401, 404, 409 o 500 según el caso.
Operaciones masivas	Rutas `/many`. Responden `{ message, data: { ok, failed, data, errors }, success, fail }` y usan 207 cuando unos elementos entran y otros no.
Borrado	Lógico por defecto (`deletedAt`, `active=false`); físico con `?hard=true`.

3.2 Listado general

La tabla siguiente es el listado completo de APIs del submódulo. Es la información que el equipo registra en la tabla de la nota compartida de Evernote (Nota 2 de la actividad).

Tabla 2. APIs del submódulo Catálogo de Catálogos.

#	Método	Ruta	Para qué sirve
1	POST	/auth/login	Iniciar sesión y elegir la base de datos (mongodb o supabase).
2	GET	/auth/session	Consultar el usuario y la base de la sesión vigente.

#	Método	Ruta	Para qué sirve
3	GET	/catalogs	Listar los catálogos con sus valores activos; filtros opcionales.
4	POST	/catalogs	Dar de alta un catálogo (puede traer sus valores iniciales).
5	POST	/catalogs/many	Dar de alta varios catálogos (arreglo de objetos).
6	PUT	/catalogs/many	Modificar varios catálogos.
7	DELETE	/catalogs/many	Eliminar varios catálogos (lógico o físico).
8	GET	/catalogs/:key	Consultar un catálogo por su llave de negocio.
9	PUT	/catalogs/:key	Modificar los datos generales de un catálogo.
10	DELETE	/catalogs/:key	Eliminar un catálogo (lógico o físico).
11	GET	/catalogs/:key/tree	Obtener el árbol de valores con sus hijos anidados.
12	GET	/catalogs/:key/values	Listar los valores, filtrando por padre o por nivel.
13	POST	/catalogs/:key/values	Agregar un valor al subdocumento / tabla detalle.
14	POST	/catalogs/:key/values/many	Agregar varios valores en una sola operación.
15	PUT	/catalogs/:key/values/many	Modificar varios valores.
16	DELETE	/catalogs/:key/values/many	Eliminar varios valores con sus descendientes.
17	GET	/catalogs/:key/values/:valueId	Consultar un valor con su nivel e hijos directos.
18	PUT	/catalogs/:key/values/:valueId	Modificar un valor.
19	DELETE	/catalogs/:key/values/:valueId	Eliminar un valor y sus descendientes.

3.3 Request y response de cada API

Para cada API se indica qué viaja en el request (header, parámetros de ruta o query y body) y qué regresa el response. Se cuidó de manera especial lo que pide la Nota 1: las APIs que insertan, modifican o eliminan un solo elemento reciben un objeto, y las que trabajan con varios reciben un arreglo de objetos o un arreglo de llaves.

Tabla 3. Request y response de las APIs de sesión y de la colección principal.

#	Request (header / parámetros / body)	Response
1	Body: `{ "usuario": "kperez", "dbServer": "supabase" }`	200: `{ token, tokenType, expiresIn, usuario, dbServer }` 400 si dbServer no es válido.
2	Header: Authorization	200: `{ usuario, dbServer, expiresAt }` 401 sin token.
3	Header: Authorization Query: `collection`, `section`, `active`	200: arreglo de catálogos, cada uno con `values[]`.
4	Header: Authorization Body (objeto): `key`, `label`, `collection` (obligatorios), `section`, `sequence`, `route`, `values[]`	201: catálogo creado. 400 key/code inválidos; 409 key repetida.
5	Body (arreglo de objetos catálogo)	201 todos creados; 207 parcial; detalle por índice en `errors`.
6	Body (arreglo): `[{ key, ...campos }]`	200 / 207 con los catálogos modificados.
7	Query: `hard` Body: `{ "keys": ["markets", "currencies"] }`	200 con el resultado por llave.
8	Ruta: `key` Query: `includeDeleted`	200: catálogo con `values[]`; 404 si no existe.
9	Ruta: `key` Body: `label`, `description`, `collection`, `section`, `sequence`, `imageUrl`, `route`, `active`	200: catálogo modificado; 400 sin campos; 404.
10	Ruta: `key` Query: `hard`	200: `{ key, deleted: "soft" | "hard", deletedAt }`; 404.

Tabla 4. Request y response de las APIs del subdocumento values[] / tabla catalog_values.

#	Request (header / parámetros / body)	Response
11	Ruta: `key`	200: catálogo con `values` anidados en `children`.
12	Ruta: `key` Query: `parentId` (uuid o `null`), `level`	200: arreglo de valores con su `level` calculado.
13	Ruta: `key` Body (objeto): `code`, `value` (obligatorios), `parentId`, `alias`, `sequence`, `description`, `imageUrl`	201: valor creado. 400 padre inexistente; 409 code o secuencia repetidos.
14	Ruta: `key` Body (arreglo de objetos valor)	201: valores creados (todo o nada).
15	Ruta: `key` Body (arreglo): `[{ id, value, alias, sequence, ... }]`	200: valores modificados.
16	Ruta: `key` Query: `hard` Body: `{ "ids": [uuid, ...] }`	200: `{ requested, affected }` con los descendientes incluidos.
17	Ruta: `key`, `valueId`	200: valor con `level` y `children`; 404.
18	Ruta: `key`, `valueId` Body: `value`, `alias`, `sequence`, `description`, `imageUrl`, `active`	200: valor modificado; 409 secuencia ocupada.
19	Ruta: `key`, `valueId` Query: `hard`	200: `{ requested, affected }`; 404.

El contrato de respuesta es único y es el mismo que definen los diseños de datos del docente para Supabase y MongoDB. Un catálogo se ve así, venga de la base que venga:

```
{
  "id": "uuid",
  "key": "strategies",
  "label": "Estrategias",
  "description": "...",
  "collection": "trading",
  "section": "Operacion",
  "sequence": 2,
  "imageUrl": null,
  "route": "/config/strategies",
  "active": true,
  "deletedAt": null,
  "createdAt": "...",
  "updatedAt": "...",
  "createdBy": "SEED",
  "updatedBy": "SEED",
  "values": [
    {
      "id": "uuid",
      "parentId": null,
      "code": "OPTIONS",
      "value": "Opciones",
      "alias": "OPT",
      "sequence": 1,
      "imageUrl": null,
      "description": null,
      "active": true,
      "deletedAt": null,
      "createdAt": "...",
      "updatedAt": "...",
      "createdBy": "SEED",
      "updatedBy": "SEED"
```

```
    }
  ]
}
```



4. Estructura base del proyecto final

El inciso B pide verificar la aplicación de la actividad anterior y prepararla para integrar el submódulo. Se usó la misma app_restful_express de la T1-Actividad 3: ya tenía separación por capas, configuración centralizada, manejo de errores y Swagger. Se respetó todo lo existente (el Catálogo de Institutos sigue funcionando igual) y se agregaron las piezas del submódulo.

4.1 Qué se verificó y qué se agregó

Tabla 5. Revisión de la aplicación base y cambios para el proyecto final.

Elemento	Estado en la T1	Cambio en esta actividad
Capas modelo / servicio / controlador / ruteo	Correcto	Se agrega una capa de repositorios entre servicio y base de datos.
config.js (punto único de variables)	Solo MongoDB	Variables de Supabase, colección de catálogos y JWT.
Conexión a base de datos	MongoDB	Se suma el pool de PostgreSQL de Supabase.
Sesión	No existía	Login que elige la base y middleware que la aplica.
Manejo de errores	Mongoose y Boom	Se agregan los códigos de error de PostgreSQL.
Swagger	Institutos	Tags Sesión, Catálogos y Valores con seguridad Bearer.

Elemento	Estado en la T1	Cambio en esta actividad
Dependencias	express, mongoose, boom...	pg (driver de PostgreSQL) y `jsonwebtoken`.

4.2 Arquitectura del submódulo

La decisión central fue no escribir dos versiones de cada API. El controlador y el servicio son únicos; lo único que cambia según la base elegida es el repositorio, que es la pieza que realmente habla con MongoDB o con Supabase. Los dos repositorios exponen exactamente las mismas funciones, así que el servicio no sabe con cuál está trabajando.

```
Petición HTTP + token
   └─ ruta (catalogs.routes.js) — requireSession: lee dbServer del token
        └─ controlador (catalogs.controller.js): req -> ctx { dbServer, usuario }
             └─ servicio (catalogs.service.js): reglas de negocio
                  └─ getCatalogsRepository(ctx.dbServer)
                       ├─ mongodb.repository.js  -> colección catalogs (values[] embebido)
                       └─ supabase.repository.js -> tablas catalogs + catalog_values (recursive
CTE)
```

This is the Repository pattern: the business rules depend on an interface, not on a database. Adding a third engine later would only require a new repository file with the same functions; services, controllers and routes would stay untouched.

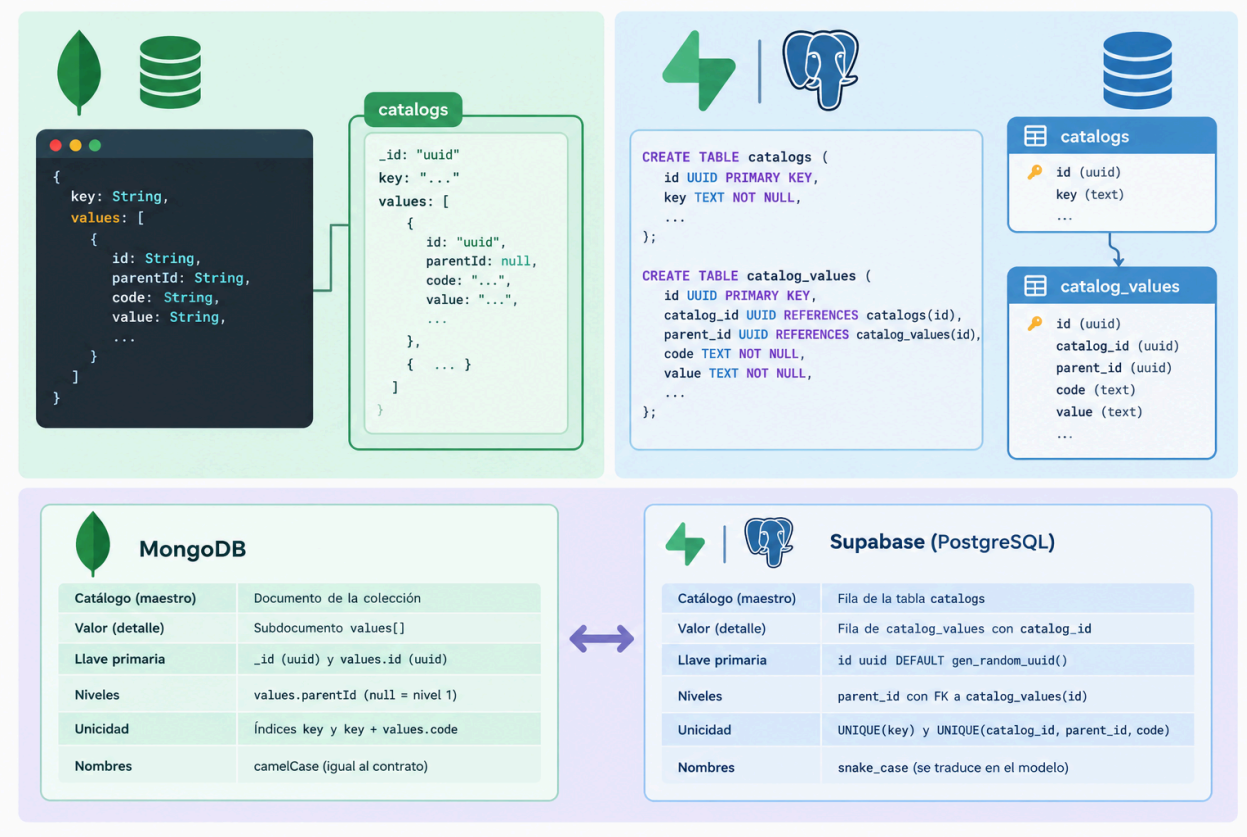
4.3 Estructura de carpetas

Los archivos nuevos se ubicaron en la misma estructura por versión y por capa que ya usaba el proyecto. Los modelos se separaron por motor y los repositorios por submódulo:

```
app_restful_express
  .env.example
  request/catalogs.http              <- ejemplos para REST Client
  scripts/
    seedCatalogs.js                  <- carga los mismos catálogos en ambas bases
    testCatalogs.js                  <- 66 peticiones de prueba
    supabase/01_catalogs.sql         <- DDL de las tablas en Supabase
  seed/catalogs.json
  src/
    index.js  app.js
    config/ config.js  database.config.js  supabase.config.js
    api/v1/
      models/mongodb/Catalogs.js     models/supabase/Catalogs.js
      repositories/catalogs/ index.js  mongodb.repository.js  supabase.repository.js
      services/   catalogs.service.js  auth.service.js
      controllers/ catalogs.controller.js  auth.controller.js
      middlewares/ session.handler.js  error.handler.js  resp.handler.js
      routes/     index.js  catalogs.routes.js  auth.routes.js
```

Las dependencias nuevas se instalaron dentro de la carpeta del proyecto:

```
npm i pg jsonwebtoken
```



5. Clases modelo

El inciso C pide las clases modelo de todas las APIs del submódulo. Como cada base guarda los datos de forma distinta, hay una clase modelo por motor, pero las dos describen la misma información:

Tabla 6. Equivalencia del modelo entre MongoDB y Supabase.

Concepto	MongoDB	Supabase (PostgreSQL)
Catálogo (maestro)	Documento de la colección `catalogs`	Fila de la tabla `catalogs`
Valor (detalle)	Elemento del subdocumento `values[]`	Fila de `catalog_values` con `catalog_id`
Llave primaria	`_id` (uuid) y `values.id` (uuid)	`id uuid DEFAULT gen_random_uuid()`
Niveles	`values.parentId` (null = nivel 1)	`parent_id` con FK a `catalog_values(id)`
Unicidad	Índices `key` y `key + values.code`	`UNIQUE(key)` y `UNIQUE(catalog_id, parent_id, code)`
Nombres	camelCase (igual al contrato)	snake_case (se traduce en el modelo)

5.1 Clase modelo en MongoDB

Es un esquema de Mongoose con el subdocumento embebido. Se usa uuid en lugar de ObjectId para que los ids sean del mismo tipo que en Supabase; así un cliente puede cambiar de base sin cambiar su forma de identificar valores.

```
// Subdocumento: un valor del catalogo. parentId arma los N niveles.
const valueSchema = new mongoose.Schema(
  {
    id: { type: String, default: randomUUID }, // uuid, igual que en Supabase
    parentId: { type: String, default: null }, // null = nivel 1
    code: { type: String, required: true, trim: true }, // UPPER_SNAKE: 'VEH_PICKUP'
    value: { type: String, required: true, trim: true }, // 'Pick-Up'
    alias: { type: String, trim: true, default: null },
    sequence: { type: Number, required: true, default: 0 },
    imageUrl: { type: String, default: null },
    description: { type: String, default: null },
    active: { type: Boolean, default: true },
    deletedAt: { type: Date, default: null },
    createdBy: { type: String, default: null },
    updatedBy: { type: String, default: null },
  },
  { _id: false, timestamps: true },
);
```

El documento raíz declara los índices del diseño (mon-1 a mon-4). El nombre de la colección viene de la variable de entorno CATALOGS_COLLECTION:

```
// Indices del diseno (mon-1 ... mon-4)
catalogSchema.index({ key: 1 }, { unique: true });
catalogSchema.index({ 'values.parentId': 1 });
catalogSchema.index({ key: 1, 'values.code': 1 }, { unique: true });
catalogSchema.index({ key: 1, 'values.sequence': 1 });

export default mongoose.model('catalogs', catalogSchema, config.CATALOGS_COLLECTION);
```

5.2 Clase modelo en Supabase

En Supabase la estructura se crea con SQL. El script `scripts/supabase/01_catalogs.sql` se ejecuta una vez en el SQL Editor del proyecto y crea las dos tablas con sus llaves, restricciones e índices:

```
-- Detalle auto-referenciado: N niveles mediante parent_id (NULL = nivel 1)
CREATE TABLE IF NOT EXISTS catalog_values (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  catalog_id  UUID NOT NULL REFERENCES catalogs(id) ON DELETE CASCADE,
  parent_id   UUID REFERENCES catalog_values(id) ON DELETE CASCADE,
  code        TEXT NOT NULL,
  value       TEXT NOT NULL,
  alias       TEXT,
  sequence    INTEGER NOT NULL DEFAULT 0,
  image_url   TEXT,
```

```
  description TEXT,
  active      BOOLEAN NOT NULL DEFAULT TRUE,
  deleted_at  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by  TEXT,
  updated_by  TEXT,
  CONSTRAINT catalog_values_code_scope UNIQUE (catalog_id, parent_id, code),
  CONSTRAINT catalog_values_seq_scope  UNIQUE (catalog_id, parent_id, sequence)
);

CREATE INDEX IF NOT EXISTS idx_catalog_values_catalog ON catalog_values (catalog_id);
CREATE INDEX IF NOT EXISTS idx_catalog_values_parent  ON catalog_values (parent_id);
```

Del lado de Node, la clase modelo describe cada tabla y la traducción entre columnas y propiedades del contrato. Las funciones `fromRow` y `toColumns` son las que permiten que la API responda siempre en camelCase:

```
export const CatalogValueTable = {
  name: 'catalog_values',
  columns: {
    id: 'id',
    catalog_id: 'catalogId',
    parent_id: 'parentId',
    code: 'code',
    value: 'value',
    alias: 'alias',
    sequence: 'sequence',
    image_url: 'imageUrl',
    description: 'description',
    active: 'active',
    deleted_at: 'deletedAt',
    created_at: 'createdAt',
    updated_at: 'updatedAt',
    created_by: 'createdBy',
    updated_by: 'updatedBy',
  },
};

// Fila SQL -> objeto camelCase (solo columnas conocidas)
export const fromRow = (table, row) =>
  Object.fromEntries(
    Object.entries(table.columns)
      .filter(([column]) => row[column] !== undefined)
      .map(([column, prop]) => [prop, row[column]]),
  );

// Objeto camelCase -> { columna: valor } (ignora lo que no sea columna)
export const toColumns = (table, data) => {
```

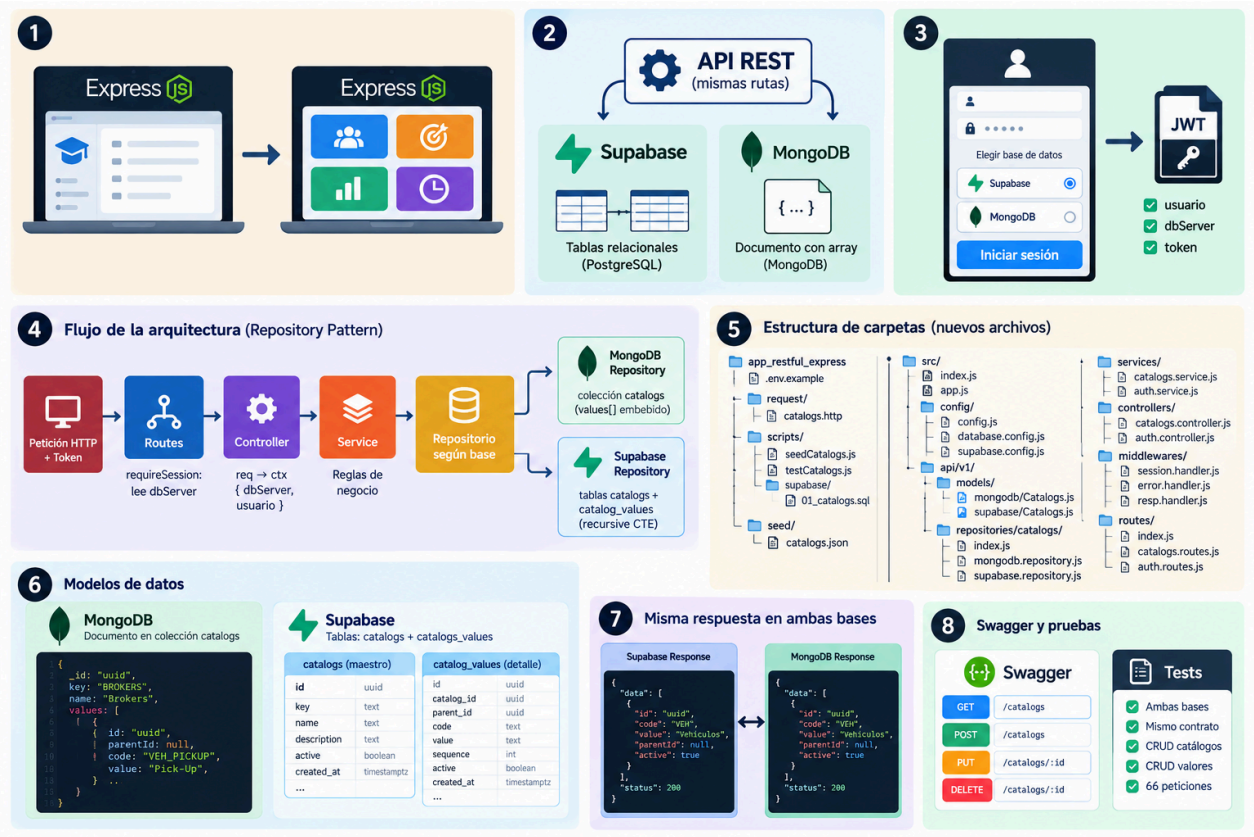
eeducation | Rows | Structure | Indexes | Constraints | Query | History | Activity | Connection | Connect | Disconnect

Filter database objects

public
- Tables 9
  - bitacora
  - catalog_values
  - catalogs
  - institutos
  - prod_serv
  - prod_serv_estatus
  - prod_serv_info_ad
  - prod_serv_presenta
  - prod_serv_presenta_archivos
- Views 3
- Materialized Views 0
- Functions 37
- Sequences 1

column_name	data_type	is_nullable	character_maximum_length	character_set_catalog	column_default	comment
id	uuid	NO	NULL	NULL	gen_random_uuid()	NULL
catalog_id	uuid	NO	NULL	NULL	NULL	NULL
parent_id	uuid	YES	NULL	NULL	NULL	NULL
code	text	NO	NULL	NULL	NULL	NULL
value	text	NO	NULL	NULL	NULL	NULL
alias	text	YES	NULL	NULL	NULL	NULL
sequence	integer	NO	NULL	NULL	0	NULL
image_url	text	YES	NULL	NULL	NULL	NULL
description	text	YES	NULL	NULL	NULL	NULL
active	boolean	NO	NULL	NULL	true	NULL
deleted_at	timestamp with time zone	YES	NULL	NULL	NULL	NULL
created_at	timestamp with time zone	NO	NULL	NULL	now()	NULL
updated_at	timestamp with time zone	NO	NULL	NULL	now()	NULL
created_by	text	YES	NULL	NULL	NULL	NULL
updated_by	text	YES	NULL	NULL	NULL	NULL

Figura 1. Estructura de la tabla catalog_values en PostgreSQL (mismo motor que Supabase).



6. Conexiones a Supabase y MongoDB con variables de entorno

El inciso D pide configurar las conexiones a la base SQL (Supabase) y NoSQL (MongoDB) en las variables de entorno y que las APIs operen con la base seleccionada al iniciar sesión. Ninguna credencial está escrita en el código.

6.1 Variables de entorno

Al .env de la T1 se agregaron las variables del submódulo. El repositorio solo contiene el .env.example, sin datos reales:

```
# --- T2-Actividad 2: Catalogo de Catalogos (proyecto final) ---
# Coleccion de MongoDB con los catalogos y sus valores embebidos
CATALOGS_COLLECTION=catalogs

# --- Base de datos SQL: Supabase (PostgreSQL) ---
# Supabase > Project Settings > Database > Connection string > URI
# Ejemplo Supabase:
postgresql://postgres.<ref>:<password>@aws-0-<region>.pooler.supabase.com:6543/postgres
# Postgres local con Docker: postgres://lab:lab_local_2026@localhost:5433/eeducation
SUPABASE_DB_URL=postgresql://postgres.<ref>:<password>@<host>:6543/postgres
SUPABASE_DB_SSL=true

# --- Sesion (JSON Web Token) ---
# Palabra secreta para firmar el token que guarda la base elegida al iniciar sesion
JWT_SECRET=<cadena-larga-y-aleatoria>
JWT_EXPIRES_IN=8h
```

Tabla 7. Variables de entorno agregadas en esta actividad.

Variable	Uso
CATALOGS_COLLECTION	Nombre de la colección de MongoDB con los catálogos.
SUPABASE_DB_URL	Cadena de conexión de Supabase (Project Settings > Database > Connection string).
SUPABASE_DB_SSL	`true` en Supabase; `false` en el PostgreSQL local de pruebas.
JWT_SECRET	Clave para firmar el token de sesión que guarda la base elegida.
JWT_EXPIRES_IN	Duración de la sesión (8h).

6.2 Configuración centralizada

Como en la T1, el resto del código nunca lee `process.env`; usa `config.X`:

```
// --- T2-Actividad 2: Catalogo de Catalogos (proyecto final) ---
// Coleccion de MongoDB con los catalogos y sus valores embebidos
CATALOGS_COLLECTION: process.env.CATALOGS_COLLECTION || 'catalogs',

// --- SQL: Supabase (PostgreSQL) ---
// Supabase > Project Settings > Database > Connection string (URI)
SUPABASE_DB_URL: process.env.SUPABASE_DB_URL,
SUPABASE_DB_SSL: process.env.SUPABASE_DB_SSL === 'true',

// --- Sesion: el usuario elige la base de datos al iniciar sesion ---
DB_SERVERS: ['mongodb', 'supabase'],
JWT_SECRET: process.env.JWT_SECRET,
JWT_EXPIRES_IN: process.env.JWT_EXPIRES_IN || '8h',
```

6.3 Conexión a Supabase

Supabase es PostgreSQL administrado, así que se conecta con el driver `pg` y un pool de conexiones. Se eligió la cadena de conexión en lugar del cliente `supabase-js` porque el árbol

de valores se arma con un recursive CTE, que solo se puede ejecutar por SQL directo. Si la variable no existe, la aplicación sigue funcionando solo con MongoDB:

```
export const connectSupabase = async () => {
  if (!config.SUPABASE_DB_URL) {
    console.warn('Supabase no configurado (SUPABASE_DB_URL): solo estara disponible MongoDB.');
    return null;
  }

  pool = new pg.Pool({
    connectionString: config.SUPABASE_DB_URL,
    ssl: config.SUPABASE_DB_SSL ? { rejectUnauthorized: false } : false,
    max: 10,
  });

  const { rows } = await pool.query('SELECT current_database() AS db');
  console.log('Supabase (PostgreSQL) is connected to:', rows[0].db);
  return pool;
};

// Devuelve el pool o falla con un error claro si nadie configuro Supabase
export const getPool = () => {
  if (!pool) {
    const error = new Error('Supabase no esta configurado en el servidor.');
    error.code = 'SUPABASE_OFF';
    throw error;
  }
  return pool;
};
```

Las altas que tocan maestro y detalle se hacen dentro de una transacción para que no queden datos a medias:

```
export const withTransaction = async (work) => {
  const client = await getPool().connect();
  try {
    await client.query('BEGIN');
    const result = await work(client);
    await client.query('COMMIT');
    return result;
  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
};
```

6.4 Selección de la base al iniciar sesión

El login recibe el usuario y la base con la que va a trabajar, valida que sea `mongodb` o `supabase` y devuelve un token firmado que contiene esa elección:

```
export const login = ({ usuario, dbServer } = {}) => {
  if (!usuario || !/^[A-Za-z0-9_.-]{3,30}$/.test(usuario)) {
    throw boom.badRequest('usuario es obligatorio (3 a 30 caracteres, sin espacios).');
  }
  const db = String(dbServer || '').toLowerCase();
  if (!config.DB_SERVERS.includes(db)) {
    throw boom.badRequest(`dbServer debe ser uno de: ${config.DB_SERVERS.join(', ')}.`);
  }
  if (db === 'supabase' && !isSupabaseOn()) {
    throw boom.serverUnavailable('Supabase no esta configurado en el servidor.');
  }

  const session = { usuario: usuario.toUpperCase(), dbServer: db };
  const token = jwt.sign(session, secret(), { expiresIn: config.JWT_EXPIRES_IN });
  return { token, tokenType: 'Bearer', expiresIn: config.JWT_EXPIRES_IN, ...session };
};
```

Un middleware lee el token en cada petición, deja la sesión en `req.session` y agrega el encabezado `X-Db-Server` a la respuesta. Después, el repositorio adecuado se elige con una sola función:

```
export const requireSession = (req, res, next) => {
  const [scheme, token] = (req.headers.authorization || '').split(' ');
  if (scheme !== 'Bearer' || !token) {
    return next(boom.unauthorized('Falta el token. Inicia sesion en POST /auth/login.'));
  }
  try {
    req.session = verify(token);
    // Encabezado informativo: en que base de datos se resolvio la peticion
    res.setHeader('X-Db-Server', req.session.dbServer);
    return next();
  } catch (error) {
    return next(error);
  }
};

const repositories = { mongodb, supabase };

export const getCatalogsRepository = (dbServer) => {
  const repository = repositories[dbServer];
  if (!repository) throw boom.badRequest(`Base de datos '${dbServer}' no soportada.`);
  if (dbServer === 'supabase' && !isSupabaseOn()) {
    throw boom.serverUnavailable('Supabase no esta configurado en el servidor.');
  }
  return repository;
};
```

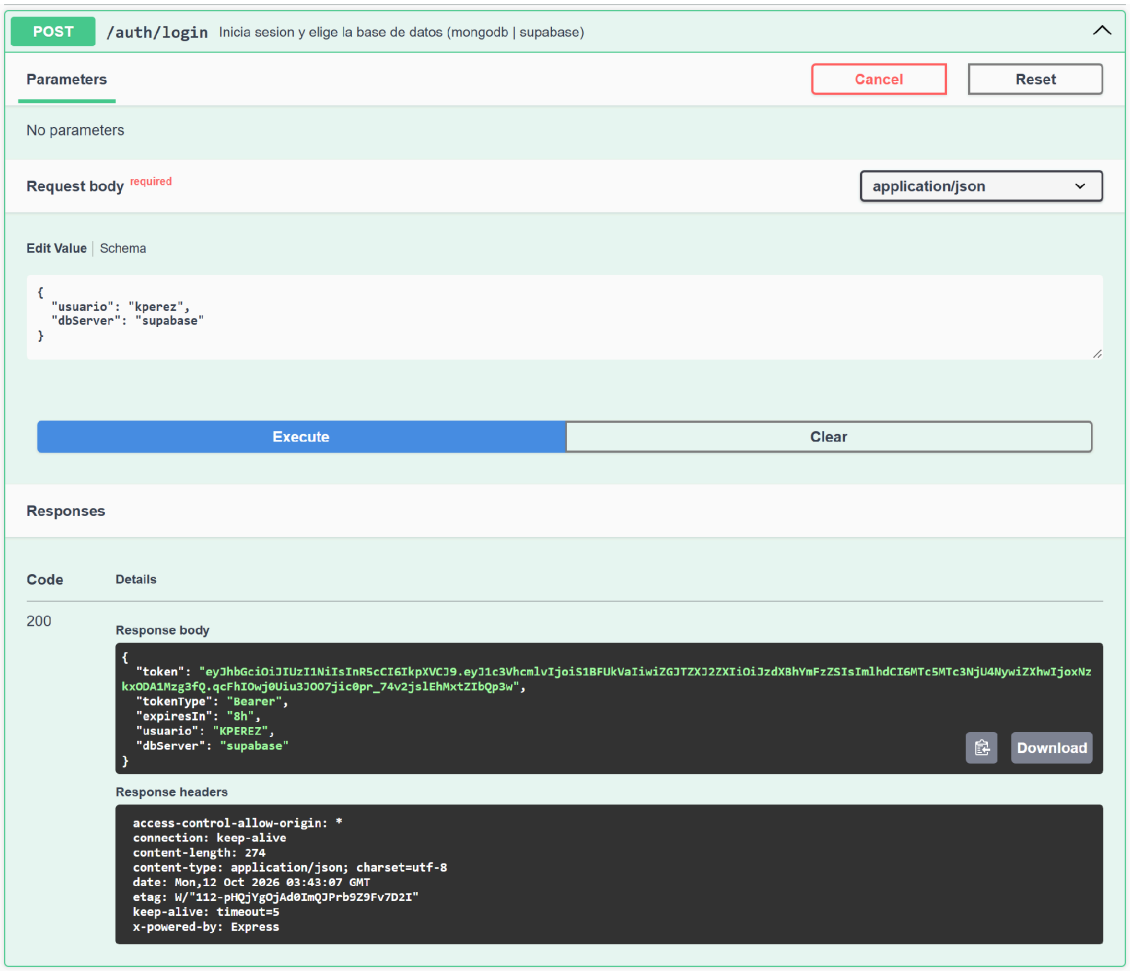


Figura 2. POST /auth/login eligiendo Supabase: el token guarda dbServer.



7. Servicios

El inciso E pide los servicios de GET, POST, PUT y DELETE, no solo para la colección principal sino para cada subdocumento. El servicio concentra las reglas de negocio y no conoce la base de datos: recibe el contexto de la sesión y le pide al repositorio que lea o escriba.

7.1 Reglas de negocio

- **Llaves con formato.** La `key` del catálogo va en snake_case y el `code` del valor en UPPER_SNAKE.
- **Unicidad.** No se repite la `key` del catálogo ni el `code` dentro del mismo catálogo (409).
- **Jerarquía válida.** El `parentId` debe apuntar a un valor activo del mismo catálogo (400).
- **Secuencia por nivel.** Si no se envía, se asigna la siguiente del mismo padre; si se envía ocupada, 409.

- **Orden padre-hijo.** Los lotes se insertan por nivel para que Supabase encuentre siempre la llave foránea.
- **Baja en cascada.** Al borrar un valor también se borran sus descendientes: no quedan hijos huérfanos.
- **Auditoría.** `createdBy` y `updatedBy` se toman del usuario de la sesión, no del body.

7.2 Servicios de la colección principal

Tabla 8. Servicios de la colección / tabla principal.

Servicio	Operación	Uno / varios
getCatalogs, getCatalog	GET	Lista y uno
createCatalog, createManyCatalogs	POST	Uno y arreglo
updateCatalog, updateManyCatalogs	PUT	Uno y arreglo
deleteCatalog, deleteManyCatalogs	DELETE	Uno y arreglo de keys

El alta valida el objeto, prepara los valores iniciales y revisa que la llave no exista, incluso si tiene baja lógica:

```
// POST /catalogs
export const createCatalog = async (ctx, payload) => {
  const catalog = prepareCatalog(payload, ctx.usuario);
  const existing = await repo(ctx).findByKey(catalog.key);
  if (existing) {
    const how = isAlive(existing) ? '' : ' (con baja logica; usa DELETE ?hard=true para
liberarla)';
    throw boom.conflict(`Ya existe un catalogo con key '${catalog.key}'${how}.`);
  }
  return present(await repo(ctx).insertCatalog(catalog));
};
```

Las operaciones masivas reutilizan la operación individual y reportan qué elementos entraron y cuáles no:

```
// Ejecuta una operacion por elemento y reporta cuales salieron bien y cuales no
const runMany = async (items, operation) => {
  const ok = [];
  const failed = [];
  for (const [index, item] of items.entries()) {
    try {
      ok.push(await operation(item));
    } catch (error) {
      failed.push({
        index,
        statusCode: boom.isBoom(error) ? error.output.statusCode : 500,
        message: error.message,
      });
    }
  }
  return { ok, failed };
};
```

7.3 Servicios del subdocumento

Tabla 9. Servicios del subdocumento values[] / tabla catalog_values.

Servicio	Operación	Uno / varios
getValues, getValue, getTree	GET	Lista filtrada, uno y árbol
addValues	POST	Uno y arreglo
updateValues	PUT	Uno y arreglo
deleteValues	DELETE	Uno y arreglo de ids

La baja de valores calcula primero los descendientes y después aplica la baja lógica o física a todos:

```
// DELETE /catalogs/:key/values/:valueId  y  /values/many { ids: [...] }
// Borrar un valor arrastra a sus descendientes: no pueden quedar hijos huerfanos
export const deleteValues = async (ctx, key, ids, { hard = false } = {}) => {
  assertArray(ids, 'ids');
  const catalog = await findCatalogOrFail(ctx, key);
  ids.forEach((id) => findValueOrFail(catalog, id));

  const affected = [...new Set(ids.flatMap((id) => [id, ...descendantsOf(catalog.values, id)]))];

  if (hard) {
    await repo(ctx).deleteValues(catalog.id, affected);
  } else {
    const now = new Date();
    await repo(ctx).updateValues(
      catalog.id,
      affected.map((id) => ({
        id,
        fields: { active: false, deletedAt: now, updatedAt: now, updatedBy: ctx.usuario },
      })),
    );
  }
  return { key: catalog.key, deleted: hard ? 'hard' : 'soft', requested: ids, affected };
};
```

7.4 Repositorios: donde sí se nota cada motor

En MongoDB, agregar valores es un `$push` al subdocumento y modificarlos es un `$set` posicional, todo en un viaje:

```
export const insertValues = async (catalogId, values) => {
  await Catalogs.updateOne(
    { _id: catalogId },
    { $push: { values: { $each: values } } },
    { runValidators: true },
  );
};

// changes: [{ id, fields }] -> un $set posicional por valor, en un solo viaje
export const updateValues = async (catalogId, changes) => {
  if (changes.length === 0) return;
```

```
  await Catalogs.bulkWrite(
    changes.map(({ id, fields }) => ({
      updateOne: {
        filter: { _id: catalogId, 'values.id': id },
        update: {
          $set: Object.fromEntries(
            Object.entries(fields).map(([prop, value]) => [`values.$.${prop}`, value]),
          ),
        },
      },
    })),
  );
};
```

En Supabase, el repositorio arma el arreglo `values` con un recursive CTE que recorre el árbol desde el nivel 1 y ordena cada rama por su secuencia, como pide el diseño del docente:

```
const VALUES_TREE_SQL = `
  WITH RECURSIVE tree AS (
    SELECT v.*, 1 AS level, ARRAY[v.sequence] AS path
      FROM catalog_values v
     WHERE v.catalog_id = ANY($1::uuid[]) AND v.parent_id IS NULL
    UNION ALL
    SELECT v.*, t.level + 1, t.path || v.sequence
      FROM catalog_values v
      JOIN tree t ON v.parent_id = t.id
  )
  SELECT * FROM tree ORDER BY catalog_id, path`;
```

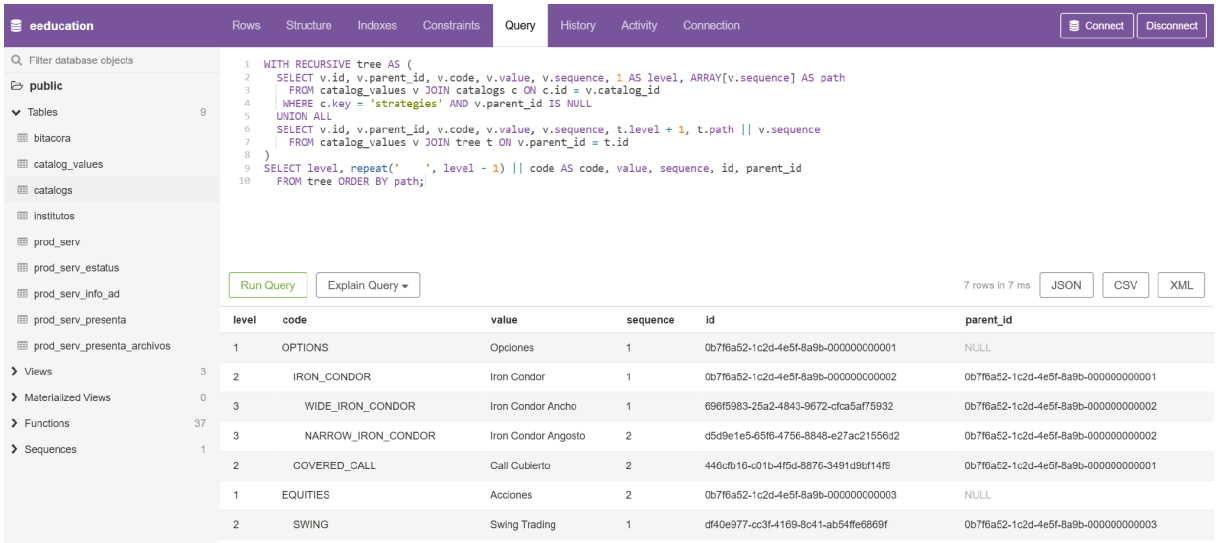
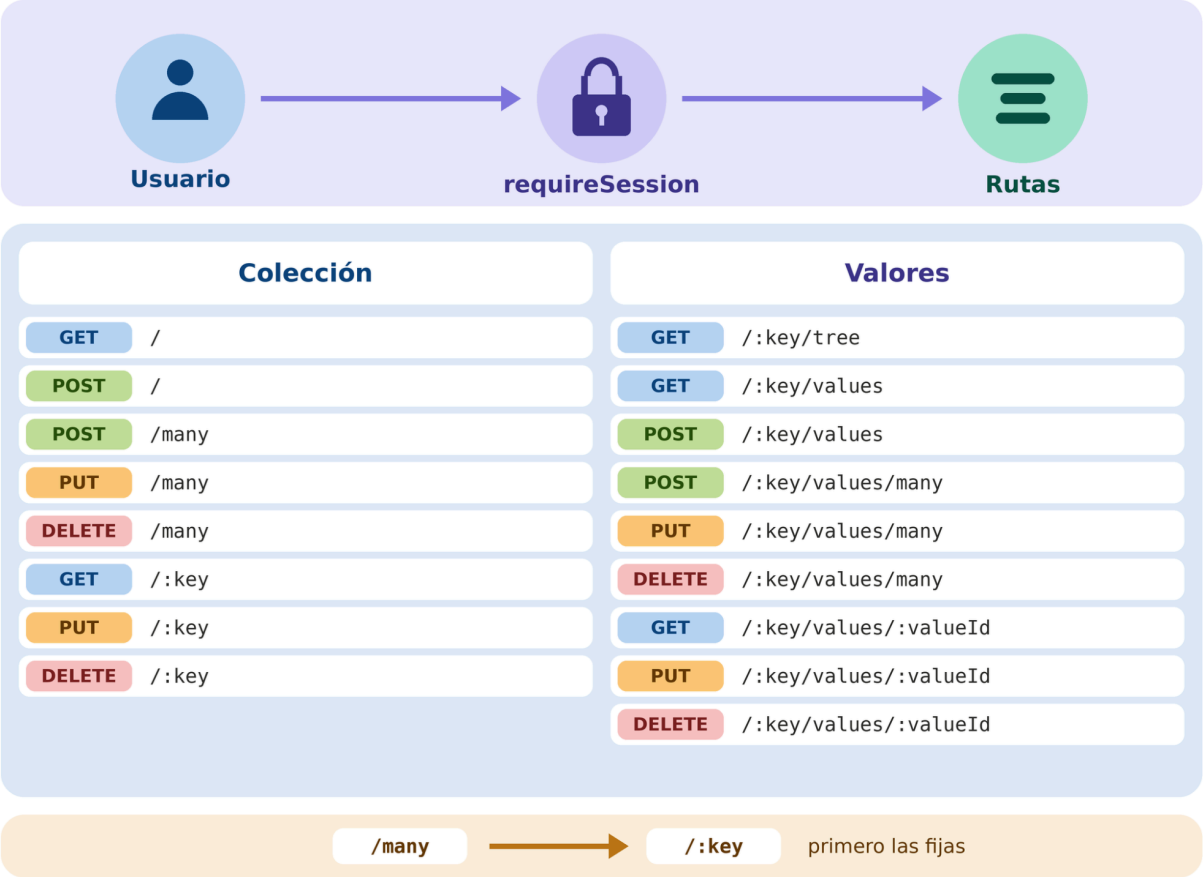


Figura 3. El recursive CTE del repositorio ejecutado sobre catalog_values: tres niveles del catálogo strategies.



8. Controladores

El inciso F pide los controladores de las cuatro operaciones para la colección y el subdocumento. El controlador traduce entre HTTP y el servicio: arma el contexto con la sesión, llama al servicio y elige el código de estado. Para no repetir try/catch en 17 métodos, se usa un envoltorio `handle`:

```
const ctxOf = (req) => ({ dbServer: req.session.dbServer, usuario: req.session.usuario });
const isHard = (req) => req.query.hard === 'true';

// Respuesta de operaciones masivas: 201/200 si todo salio, 207 si hubo de todo,
// y el codigo del primer error si no salio ninguno.
const sendMany = (res, { ok, failed }, successCode, label) => {
  const body = { ok: ok.length, failed: failed.length, data: ok, errors: failed };
  if (failed.length === 0) return res.status(successCode).json(OK(`${label}: ${ok.length}`,
body));
  if (ok.length === 0) {
    return res.status(failed[0].statusCode).json(FAIL(`${label}: ninguno`, body));
  }
  return res.status(207).json(FAIL(`${label}: ${ok.length} de ${ok.length + failed.length}`,
body));
};

// Envuelve cada controlador en try/catch y manda los errores al middleware
```

```
const handle = (fn) => async (req, res, next) => {
  try {
    await fn(req, res);
  } catch (error) {
    next(error);
  }
};

// ========================== CATALOGOS ====================================

export const getCatalogs = handle(async (req, res) => {
  res.status(200).json(await CatalogsServices.getCatalogs(ctxOf(req), req.query));
});
```

Los controladores del subdocumento reciben un objeto en las rutas individuales y un arreglo en las rutas `/many`:

```
export const postValue = handle(async (req, res) => {
  const [created] = await CatalogsServices.addValues(ctxOf(req), req.params.key, [req.body]);
  res.status(201).json(created);
});

export const postManyValues = handle(async (req, res) => {
  const created = await CatalogsServices.addValues(ctxOf(req), req.params.key, req.body);
  res.status(201).json(OK(`Valores agregados: ${created.length}`, created));
});

export const putValue = handle(async (req, res) => {
  const change = { ...req.body, id: req.params.valueId };
  const [updated] = await CatalogsServices.updateValues(ctxOf(req), req.params.key, [change]);
  res.status(200).json(updated);
});
```

Tabla 10. Códigos HTTP que devuelve el submódulo.

Código	Cuándo
200	Consultas, modificaciones y bajas correctas.
201	Altas correctas (uno o varios).
207	Operación masiva con resultado parcial.
400	Datos inválidos: key o code sin formato, padre inexistente, body vacío, uuid mal formado.
401	Sin token o token expirado.
404	Catálogo o valor inexistente (o con baja lógica).
409	Key, code o secuencia repetidos.

9. Ruteos

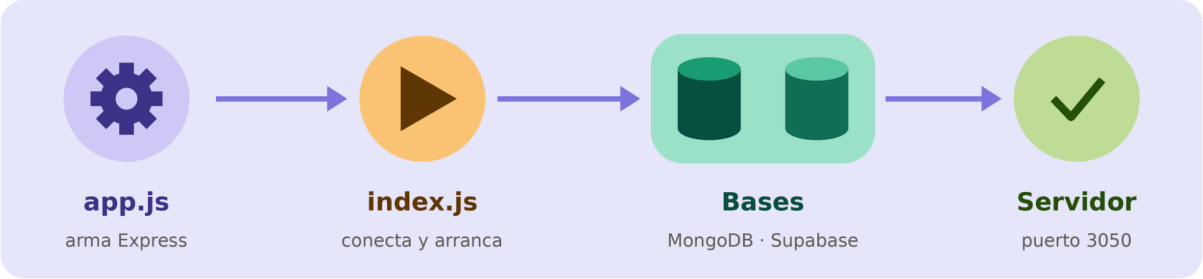
El inciso G pide los ruteos de todas las operaciones para la colección principal y para cada subdocumento. Todas las rutas de catálogos pasan primero por `requireSession`. Las rutas fijas `/many` se declaran antes que las rutas con parámetro; si no, Express tomaría la palabra "many" como si fuera una key:

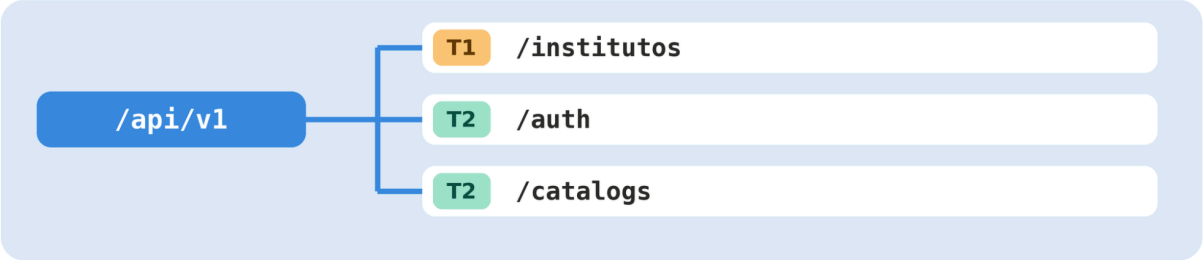
```
router.use(requireSession);
router.get('/', CatalogsController.getCatalogs);
router.post('/', CatalogsController.postCatalog);
router.post('/many', CatalogsController.postManyCatalogs);
router.put('/many', CatalogsController.putManyCatalogs);
router.delete('/many', CatalogsController.deleteManyCatalogs);
router.get('/:key', CatalogsController.getCatalog);
router.put('/:key', CatalogsController.putCatalog);
router.delete('/:key', CatalogsController.deleteCatalog);
router.get('/:key/tree', CatalogsController.getTree);
router.get('/:key/values', CatalogsController.getValues);
router.post('/:key/values', CatalogsController.postValue);
router.post('/:key/values/many', CatalogsController.postManyValues);
router.put('/:key/values/many', CatalogsController.putManyValues);
router.delete('/:key/values/many', CatalogsController.deleteManyValues);
router.get('/:key/values/:valueId', CatalogsController.getValue);
router.put('/:key/values/:valueId', CatalogsController.putValue);
router.delete('/:key/values/:valueId', CatalogsController.deleteValue);
export default router;
```

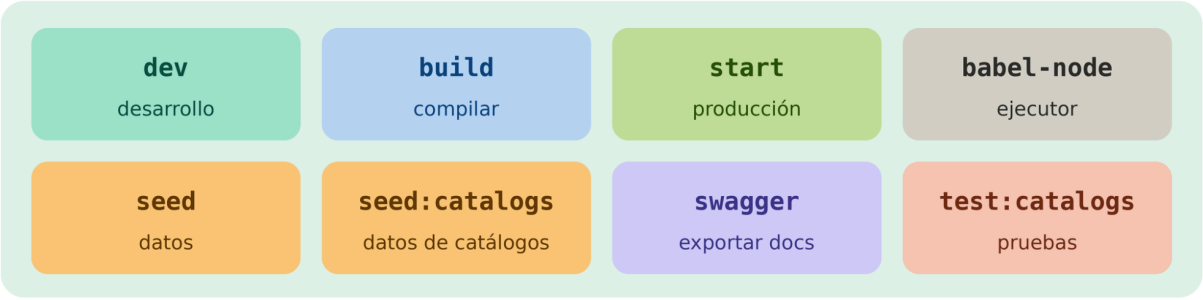
Cada ruta tiene su comentario @openapi, de modo que la documentación de Swagger se genera desde el mismo archivo. Las rutas del submódulo quedan agrupadas en tres secciones y muestran el candado de autenticación Bearer:



Figura 4. Swagger UI con las APIs de Sesión, Catálogos y Valores.







10. Configuración del Index y del App principal

El inciso H pide configurar el Index y el App para ejecutar y desplegar la aplicación. Se conservó la separación de la T1: `app.js` arma Express y `index.js` solo conecta las bases y levanta el servidor.

10.1 index.js

```
const start = async () => {
  try {
    await connectDB();
    await connectSupabase();

    app.listen(app.get('port'), () => {
      const base = `http://${config.HOST}:${app.get('port')}${config.API_URL}`;
      console.log(`Server is running on: ${base}`);
      console.log(`Catalogo de Institutos: ${base}/institutos`);
      console.log(`Catalogo de Catalogos:  ${base}/catalogs (requiere sesion:
${base}/auth/login)`);
    });
  } catch (error) {
    console.error('No se pudo iniciar el servidor:', error.message);
    process.exit(1);
  }
```

```
};

start();
```

10.2 Registro de rutas

El índice de rutas monta el submódulo junto al proceso DEMO de la T1, bajo el mismo prefijo de versión:

```
const routerAPI = (app) => {
  const router = Router();
  const api = config.API_URL;

  app.use(api, router);

  // T1-Actividad 3: proceso DEMO
  router.use('/institutos', institutosRoutes);

  // T2-Actividad 2: submodulo del proyecto final (Supabase y MongoDB)
  router.use('/auth', authRoutes);
  router.use('/catalogs', catalogsRoutes);

  return router;
};

export default routerAPI;
```

10.3 Arranque

Al ejecutar `npm run dev` la consola confirma las dos conexiones y las rutas publicadas:

```
Swagger UI en: http://localhost:3050/api/v1/api-docs
Database is connected to: db_eeducation
Supabase (PostgreSQL) is connected to: eeducation
Server is running on: http://localhost:3050/api/v1
Catalogo de Institutos: http://localhost:3050/api/v1/institutos
Catalogo de Catalogos:  http://localhost:3050/api/v1/catalogs (requiere sesion:
http://localhost:3050/api/v1/auth/login)
```

Los scripts de package.json quedaron así:

```
"scripts": {
  "babel-node": "babel-node",
  "dev": "nodemon --exec npm run babel-node -- src/index.js",
  "build": "babel src --out-dir dist",
  "start": "node dist/index.js",
  "seed": "babel-node scripts/seed.js",
  "swagger": "babel-node scripts/exportSwagger.js",
  "seed:catalogs": "babel-node scripts/seedCatalogs.js",
  "test:catalogs": "babel-node scripts/testCatalogs.js"
},
```

11. Pruebas y evidencias

11.1 Datos de prueba

`npm run seed:catalogs` carga los mismos cuatro catálogos en las dos bases, pasando por el servicio para que cumplan las mismas reglas que la API. El catálogo strategies tiene tres niveles:

```
Database is connected to: db_eeducation
Supabase (PostgreSQL) is connected to: eeducation

[mongodb]
+ brokers       3 valores
+ strategies    7 valores
+ indicators    3 valores
+ timeframes    3 valores

[supabase]
+ brokers       3 valores
+ strategies    7 valores
+ indicators    3 valores
+ timeframes    3 valores
```

11.2 Prueba automatizada en ambos motores

`npm run test:catalogs` ejecuta el mismo guion contra MongoDB y contra Supabase: las 19 APIs, las variantes de uno y de varios elementos y los casos de error. Las 66 peticiones devolvieron el código esperado:

```
OK   401 GET    /catalogs                                    sin token
OK   400 POST   /auth/login                                  dbServer invalido
OK   200 POST   /auth/login                                  [mongodb] login
OK   200 GET    /auth/session                                [mongodb] sesion
OK   200 GET    /catalogs                                    [mongodb] lista
OK   200 GET    /catalogs?collection=trading&section=Analisis [mongodb] filtro collection
OK   200 GET    /catalogs/strategies                         [mongodb] uno
OK   200 GET    /catalogs/strategies/tree                    [mongodb] arbol
OK   200 GET    /catalogs/strategies/values?level=2          [mongodb] valores nivel 2
OK   201 POST   /catalogs                                    [mongodb] alta catalogo
OK   409 POST   /catalogs                                    [mongodb] alta duplicada
OK   400 POST   /catalogs                                    [mongodb] key invalida
OK   201 POST   /catalogs/many                               [mongodb] alta masiva
OK   207 POST   /catalogs/many                               [mongodb] alta masiva parcial
OK   200 PUT    /catalogs/order_types                        [mongodb] modifica catalogo
OK   200 PUT    /catalogs/many                               [mongodb] modifica masivo
OK   201 POST   /catalogs/order_types/values                 [mongodb] alta valor
OK   201 POST   /catalogs/order_types/values                 [mongodb] alta valor nivel 2
OK   409 POST   /catalogs/order_types/values                 [mongodb] code repetido
OK   400 POST   /catalogs/order_types/values                 [mongodb] padre inexistente
OK   201 POST   /catalogs/order_types/values/many            [mongodb] alta valores masiva
OK   200 GET    /catalogs/order_types/values/{id}            [mongodb] consulta valor
OK   200 PUT    /catalogs/order_types/values/{id}            [mongodb] modifica valor
OK   200 PUT    /catalogs/order_types/values/many            [mongodb] modifica valores masivo
OK   409 PUT    /catalogs/order_types/values/{id}            [mongodb] secuencia ocupada
OK   200 DELETE /catalogs/order_types/values/{id}            [mongodb] baja logica valor (cascada)
```

```
OK   404 GET    /catalogs/order_types/values/{id}          [mongodb] valor ya borrado
OK   200 DELETE /catalogs/order_types/values/many?hard=true [mongodb] baja fisica valores masiva
OK   200 GET    /catalogs/strategies/values?parentId={id}  [mongodb] nivel 2 de strategies
OK   200 DELETE /catalogs/order_types                      [mongodb] baja logica catalogo
OK   404 GET    /catalogs/order_types                      [mongodb] catalogo borrado
OK   200 DELETE /catalogs/order_types?hard=true            [mongodb] baja fisica tras baja logica
OK   200 DELETE /catalogs/risk_levels?hard=true            [mongodb] baja fisica catalogo
OK   200 DELETE /catalogs/many?hard=true                   [mongodb] baja fisica masiva
OK   200 POST   /auth/login                                [supabase] login
OK   200 GET    /auth/session                              [supabase] sesion
OK   200 GET    /catalogs                                  [supabase] lista
OK   200 GET    /catalogs?collection=trading&section=Analisis [supabase] filtro collection
OK   200 GET    /catalogs/strategies                       [supabase] uno
OK   200 GET    /catalogs/strategies/tree                  [supabase] arbol
OK   200 GET    /catalogs/strategies/values?level=2        [supabase] valores nivel 2
OK   201 POST   /catalogs                                  [supabase] alta catalogo
OK   409 POST   /catalogs                                  [supabase] alta duplicada
OK   400 POST   /catalogs                                  [supabase] key invalida
OK   201 POST   /catalogs/many                             [supabase] alta masiva
OK   207 POST   /catalogs/many                             [supabase] alta masiva parcial
OK   200 PUT    /catalogs/order_types                      [supabase] modifica catalogo
OK   200 PUT    /catalogs/many                             [supabase] modifica masivo
OK   201 POST   /catalogs/order_types/values               [supabase] alta valor
OK   201 POST   /catalogs/order_types/values               [supabase] alta valor nivel 2
OK   409 POST   /catalogs/order_types/values               [supabase] code repetido
OK   400 POST   /catalogs/order_types/values               [supabase] padre inexistente
OK   201 POST   /catalogs/order_types/values/many          [supabase] alta valores masiva
OK   200 GET    /catalogs/order_types/values/{id}          [supabase] consulta valor
OK   200 PUT    /catalogs/order_types/values/{id}          [supabase] modifica valor
OK   200 PUT    /catalogs/order_types/values/many          [supabase] modifica valores masivo
OK   409 PUT    /catalogs/order_types/values/{id}          [supabase] secuencia ocupada
OK   200 DELETE /catalogs/order_types/values/{id}          [supabase] baja logica valor (cascada)
OK   404 GET    /catalogs/order_types/values/{id}          [supabase] valor ya borrado
OK   200 DELETE /catalogs/order_types/values/many?hard=true [supabase] baja fisica valores masiva
OK   200 GET    /catalogs/strategies/values?parentId={id}  [supabase] nivel 2 de strategies
OK   200 DELETE /catalogs/order_types                      [supabase] baja logica catalogo
OK   404 GET    /catalogs/order_types                      [supabase] catalogo borrado
OK   200 DELETE /catalogs/order_types?hard=true            [supabase] baja fisica tras baja logica
OK   200 DELETE /catalogs/risk_levels?hard=true            [supabase] baja fisica catalogo
OK   200 DELETE /catalogs/many?hard=true                   [supabase] baja fisica masiva

66/66 peticiones con el codigo esperado.
```

11.3 Evidencias con Supabase

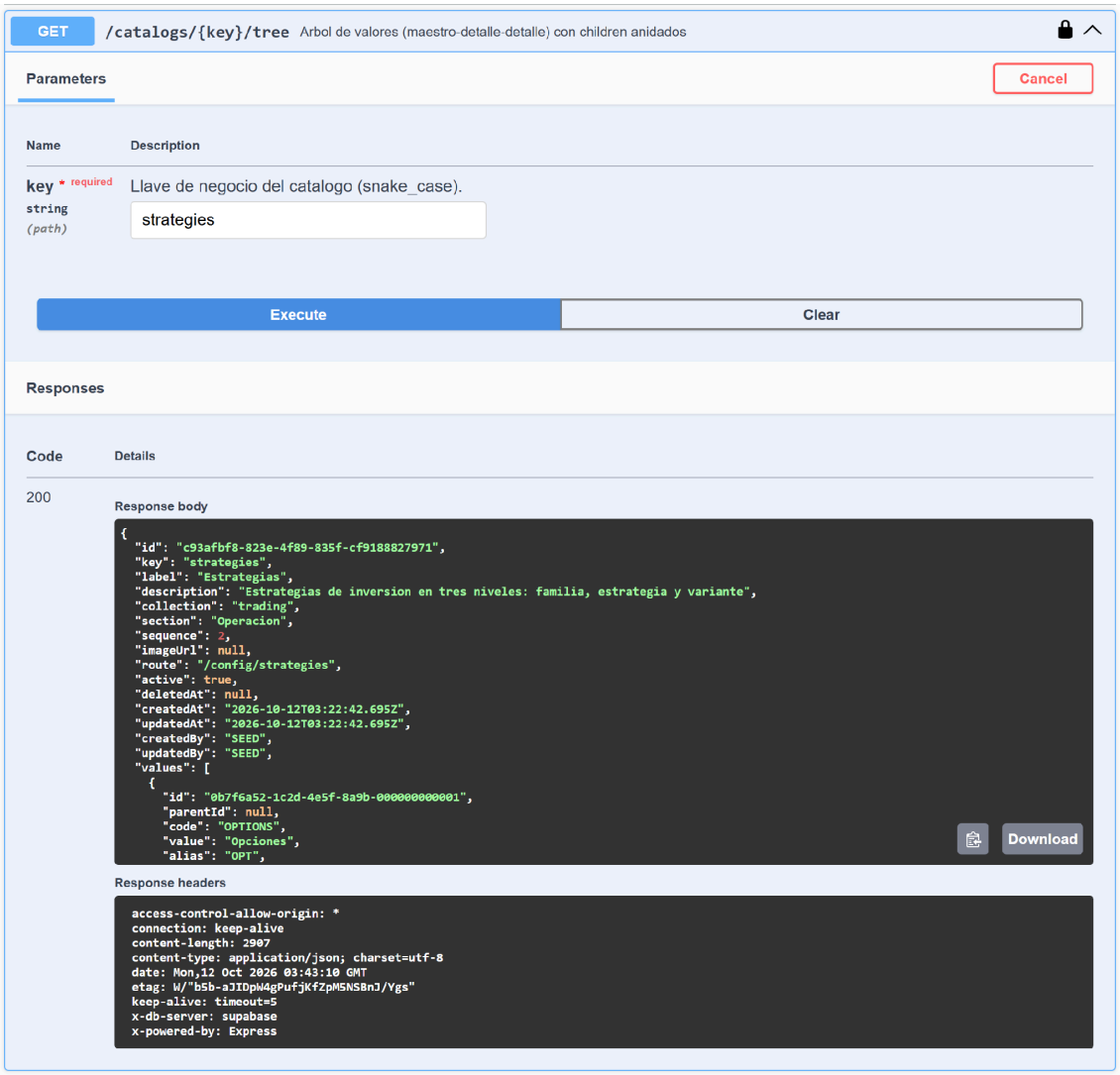


Figura 5. GET /catalogs/strategies/tree con sesión en Supabase; el encabezado x-db-server lo confirma.

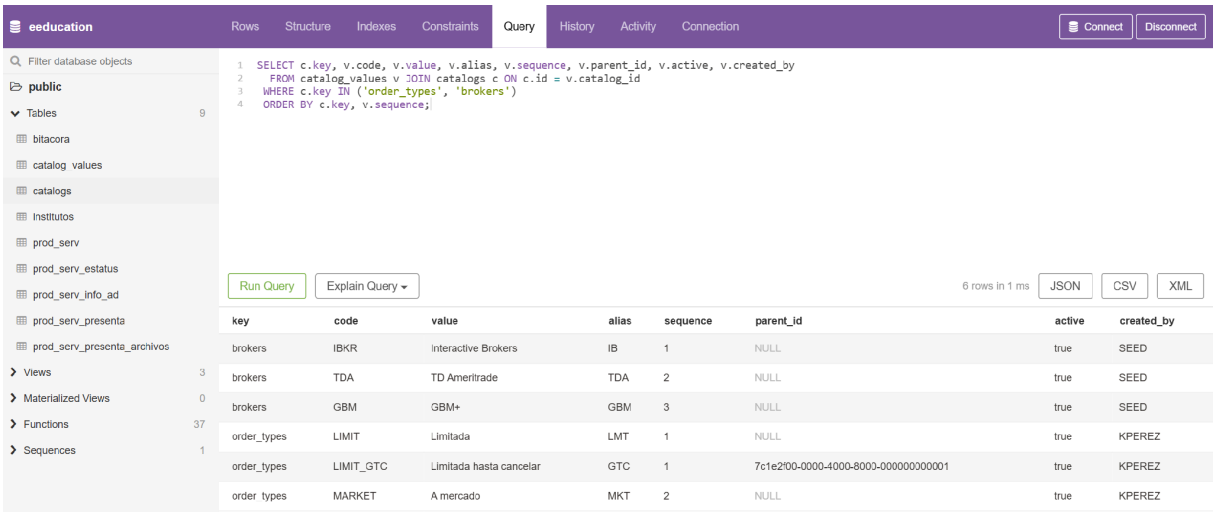


Figura 6. Filas de catalog_values: LIMIT_GTC es hijo de LIMIT a través de parent_id.

11.4 Evidencias con MongoDB

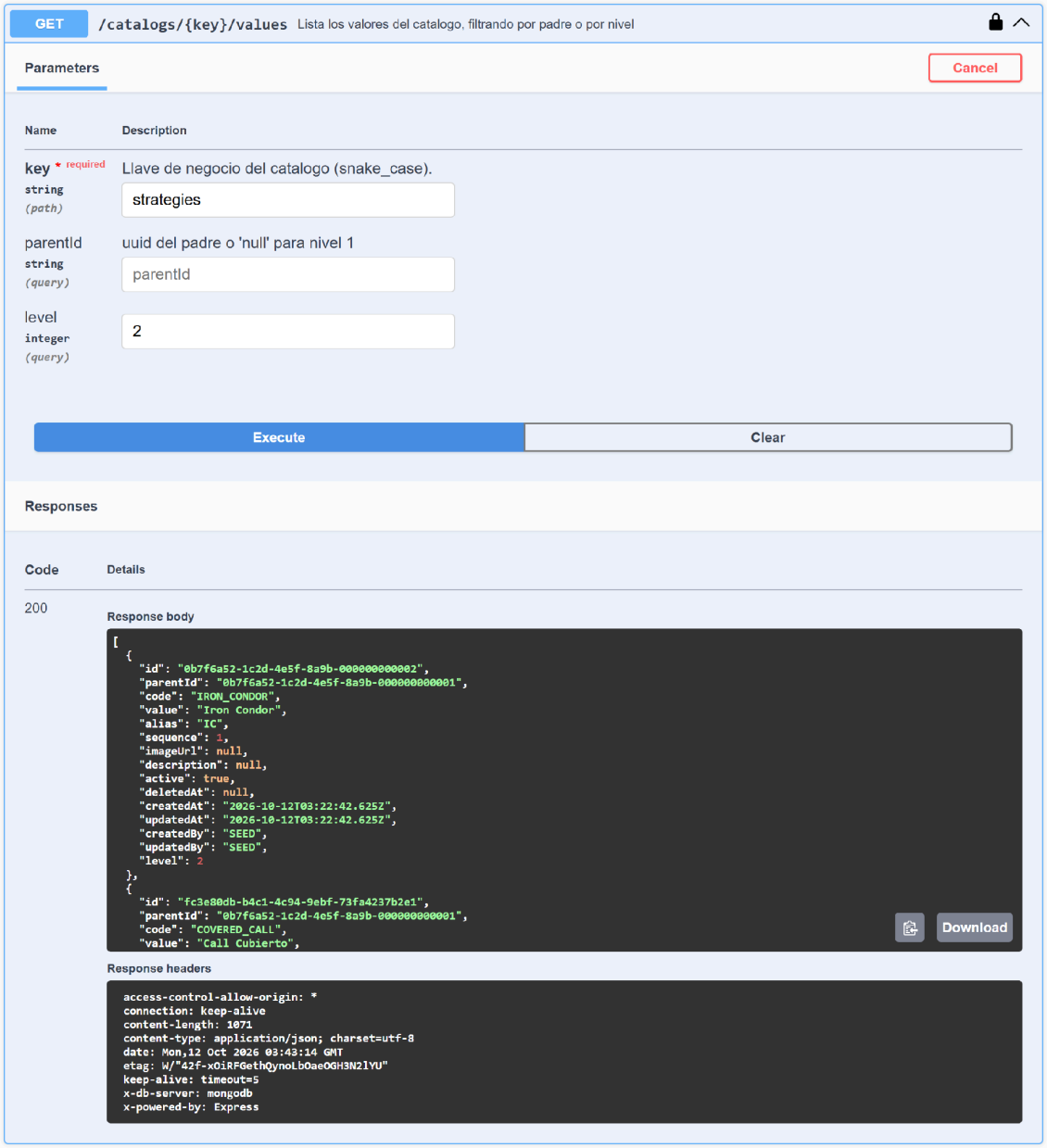


Figura 7. GET /catalogs/strategies/values?level=2: la misma API respondiendo desde MongoDB.

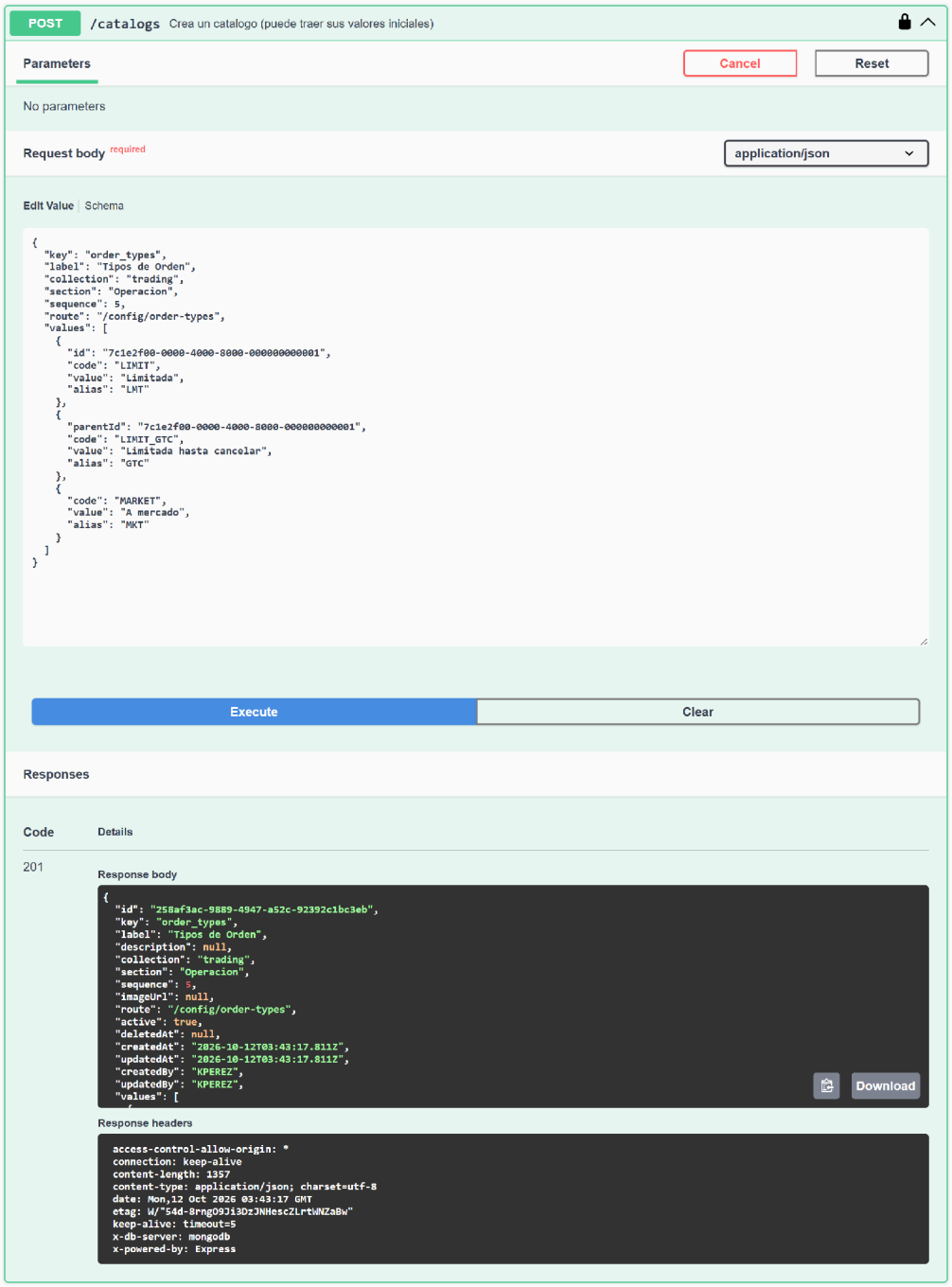


Figura 8. POST /catalogs: alta de order_types con valores de dos niveles (201).

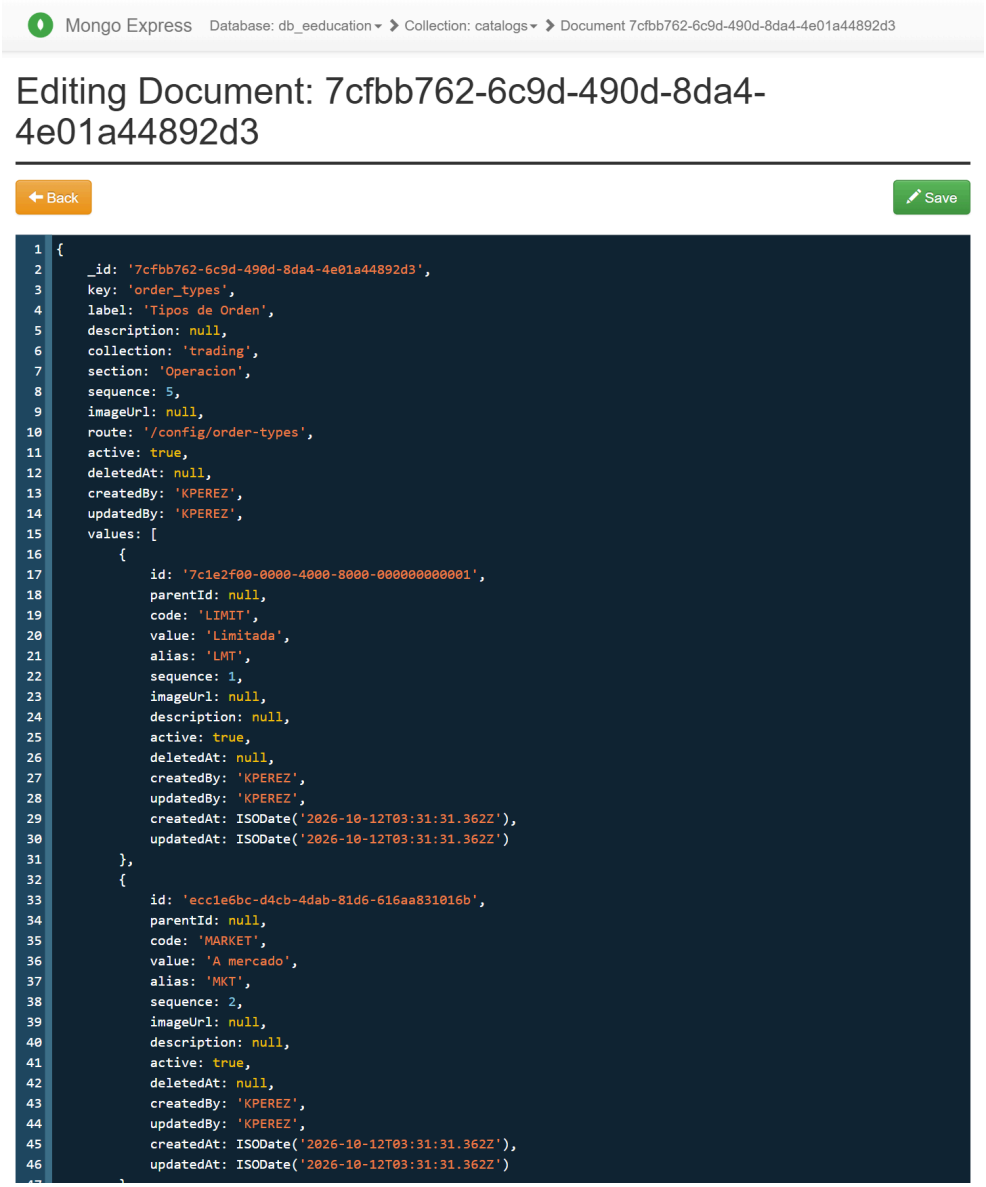


Figura 9. El documento order_types en MongoDB con su subdocumento values[] embebido.

11.5 Manejo de errores

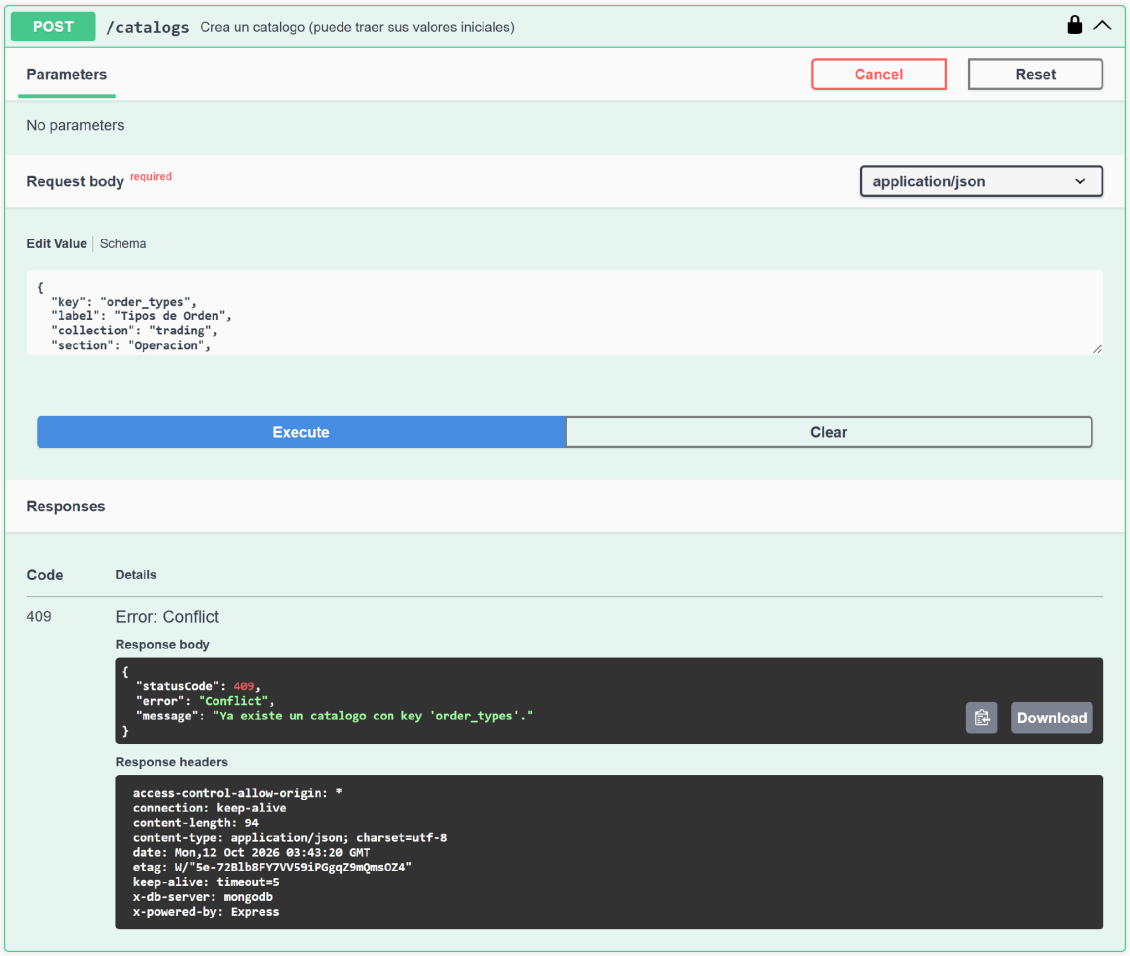


Figura 10. Alta repetida del mismo catálogo: 409 con el formato uniforme de error.

11.6 Un solo contrato para los dos motores

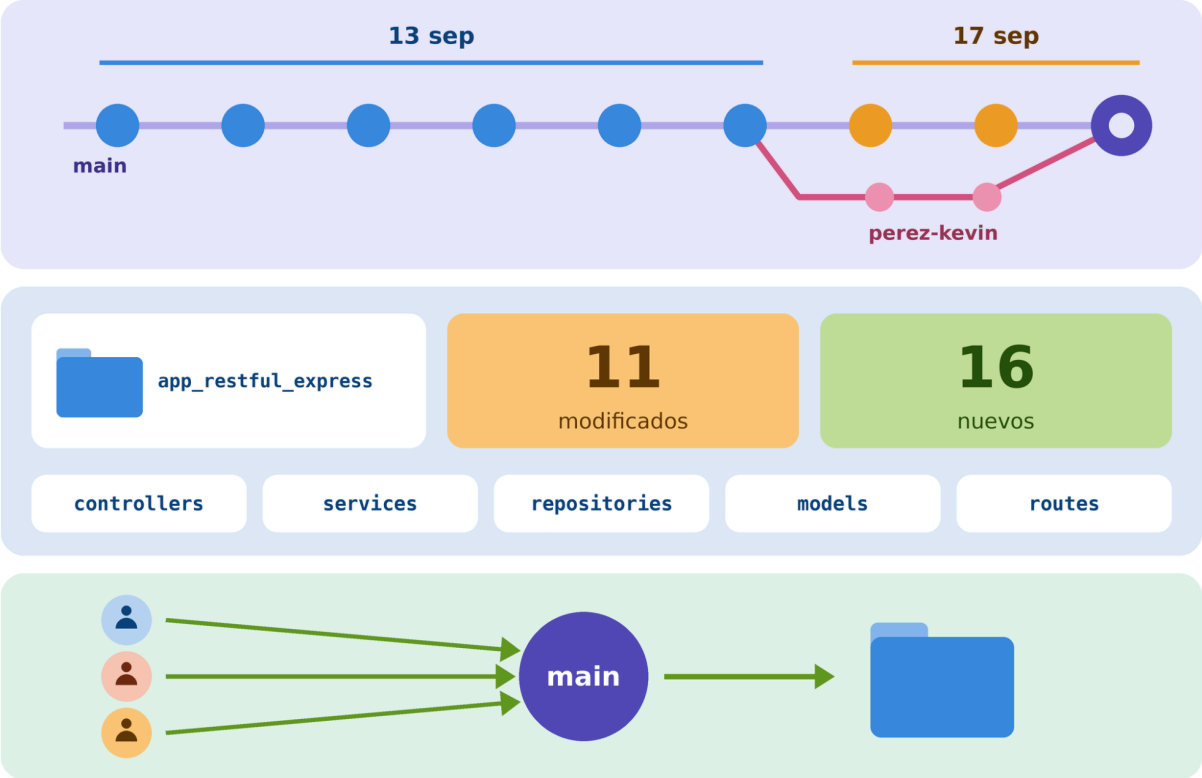
Para comprobar que la API es realmente la misma, se pidió `GET /catalogs?collection=trading` con una sesión en cada base y se compararon las respuestas sin ids ni fechas (que naturalmente cambian). El resultado fue idéntico, carácter por carácter, incluido el orden de los valores.

11.7 Problemas encontrados y cómo se resolvieron

Tabla 11. Problemas detectados durante las pruebas.

Problema	Causa	Solución
Mismos datos, distinto orden de values	MongoDB devolvía el orden de inserción y Supabase el recorrido del CTE.	El servicio ordena en profundidad (padre, hijos por secuencia) en ambos motores.
Mismos campos, distinto orden de propiedades	Mongoose agrega createdAt/updatedAt al final.	Función `ordered` que fija el orden del contrato.

Problema	Causa	Solución
No se podía borrar físicamente un catálogo con baja lógica	La baja física buscaba solo catálogos activos.	La baja física busca también los que tienen deletedAt.
Error de llave foránea al insertar hijos	En un lote el hijo podía llegar antes que el padre.	Los valores se ordenan por nivel antes de insertarlos.



12. Repositorio y bitácora

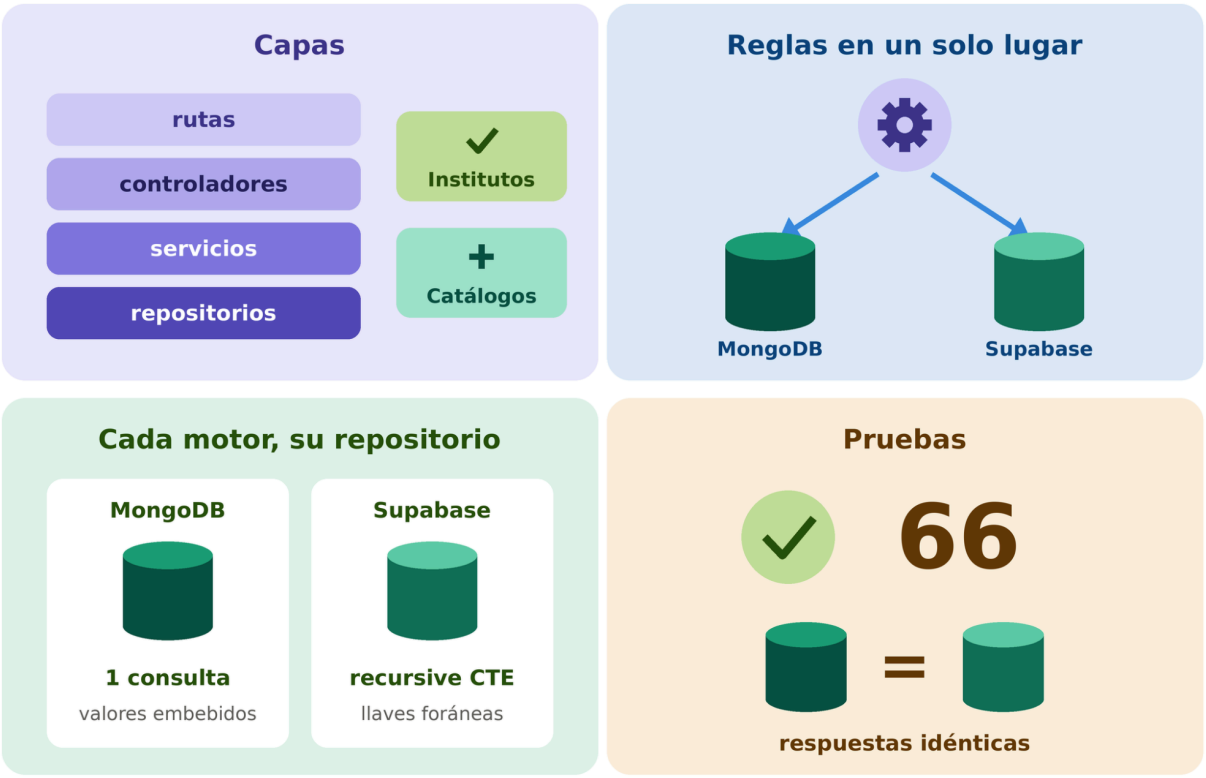
El código está en el repositorio del equipo, en la carpeta app_restful_express: **https://github.com/BetoCW/lab-servicios-web**. Cada integrante trabaja en su propia rama y los cambios se integran a main. La bitácora de aportaciones (push) del repositorio es la siguiente:

```
8bd0365 2026-09-17 BetoCW  Merge rama perez-kevin: T1-Actividad 3 (Express + SAP CDS) y README
reorganizado
ac2d642 2026-09-17 BetoCW  docs: reorganiza el README por actividad y explica cómo bajar los
cambios
c98c85e 2026-09-17 BetoCW  T1-Actividad 3: Catálogo de Institutos en Express y en SAP CDS
1e2d951 2026-09-13 BetoCW  docs: tutorial de PostgreSQL con Docker desde cero
7ec7d6a 2026-09-13 BetoCW  docs: guías paso a paso, diseño de datos y diagramas
9c643d0 2026-09-13 BetoCW  database: Docker local, esquema PostgreSQL, datos de prueba y vistas
JSON
69c3709 2026-09-13 BetoCW  eeducation-api: API de institutos y productos/servicios con detail_row
f6bd1ae 2026-09-13 BetoCW  catalogos-api: API REST del catálogo de catálogos con Express 5 y
```

```
MongoDB
690cf81 2026-09-13 BetoCW  Estructura inicial del repositorio y guía general
```

Los archivos de esta actividad que se integran en la aplicación son:

```
 M app_restful_express/.env.example
 M app_restful_express/README.md
 M app_restful_express/docs/openapi.json
 M app_restful_express/package-lock.json
 M app_restful_express/package.json
 M app_restful_express/src/api/v1/middlewares/error.handler.js
 M app_restful_express/src/api/v1/routes/index.js
 M app_restful_express/src/app.js
 M app_restful_express/src/config/config.js
 M app_restful_express/src/docs/swagger.js
 M app_restful_express/src/index.js
?? app_restful_express/request/catalogs.http
?? app_restful_express/scripts/seedCatalogs.js
?? app_restful_express/scripts/supabase/
?? app_restful_express/scripts/testCatalogs.js
?? app_restful_express/seed/catalogs.json
?? app_restful_express/src/api/v1/controllers/auth.controller.js
?? app_restful_express/src/api/v1/controllers/catalogs.controller.js
?? app_restful_express/src/api/v1/middlewares/session.handler.js
?? app_restful_express/src/api/v1/models/mongodb/
?? app_restful_express/src/api/v1/models/supabase/
?? app_restful_express/src/api/v1/repositories/
?? app_restful_express/src/api/v1/routes/auth.routes.js
?? app_restful_express/src/api/v1/routes/catalogs.routes.js
?? app_restful_express/src/api/v1/services/auth.service.js
?? app_restful_express/src/api/v1/services/catalogs.service.js
?? app_restful_express/src/config/supabase.config.js
```



13. Conclusiones

Integrar el submódulo en la aplicación de la T1 confirmó que la separación por capas no era un formalismo: el Catálogo de Institutos siguió funcionando sin tocarlo, y el Catálogo de Catálogos se agregó como un módulo más, con sus propios modelos, servicios, controladores y rutas.

El requisito de trabajar con Supabase y MongoDB a la vez fue lo que más decisiones obligó a tomar. Escribir cada API dos veces habría duplicado el código y, sobre todo, las reglas de negocio. Con la capa de repositorios, las reglas viven en un solo servicio y cada motor solo resuelve cómo guardar y leer. La selección de la base en el login hace que el cambio sea transparente para el cliente: las mismas rutas, el mismo JSON, otra base.

Las diferencias entre los motores sí aparecieron, pero quedaron encerradas en su repositorio. En MongoDB el catálogo completo se lee en una sola consulta porque los valores están embebidos; en Supabase hizo falta un recursive CTE y transacciones, a cambio de llaves foráneas y restricciones que la base garantiza por sí misma.

Por último, las pruebas volvieron a demostrar su valor: los dos motores respondían 200 desde el principio, pero solo comparando el contenido se descubrió que el orden de los valores no coincidía. Hoy las 66 peticiones pasan y las respuestas de ambas bases son idénticas.

Referencias

Auth0. (2026). JSON Web Tokens introduction. https://jwt.io/introduction

Automattic. (2026). Mongoose documentation: Subdocuments. https://mongoosejs.com/docs/subdocs.html

Brianc. (2026). node-postgres documentation. https://node-postgres.com/

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

MongoDB, Inc. (2026). Embedded data versus references. https://www.mongodb.com/docs/manual/data-modeling/

OpenJS Foundation. (2026). Express 4.x API reference. https://expressjs.com/en/4x/api.html

PostgreSQL Global Development Group. (2026). WITH queries (Common Table Expressions). https://www.postgresql.org/docs/current/queries-with.html

SmartBear Software. (2026). OpenAPI Specification 3.0.3. https://swagger.io/specification/v3/

Supabase. (2026). Connecting to your database. https://supabase.com/docs/guides/database/connecting-to-postgres